

Lexical Structure & Scanning

Liting Zhang

University of West Florida

- 1 Overview and Goals
- 2 Lexical Basics
- 3 Regular Expressions
- 4 Finite Automata
- 5 From Regex to a Scanner
- 6 Practice: Tokenizing Code
- 7 Implementation in JavaScript
- 8 Wrap-up

Overview

How raw characters become tokens.

- What lexical analysis is
- Characters, lexemes, tokens
- Token categories (Java, C++, JavaScript)
- Regular expressions and a bit of theory
- How a scanner (lexer) is designed
- A tiny lexer in JavaScript (Node)
- How to handle whitespace, comments, and errors
- How this connects to Project 1

Goals for Week 2

After this week, you should be able to:

- explain what characters, lexemes, and tokens are
- describe where lexical analysis fits in a compiler or interpreter
- list common token categories in languages like Java, C++, JavaScript
- write simple regular expressions for typical token patterns
- explain how finite automata relate to regular expressions
- read and understand a small hand-written scanner (lexer)
- run and lightly modify a tiny TypeScript lexer (for Project 1)

- 1 Overview and Goals
- 2 Lexical Basics**
- 3 Regular Expressions
- 4 Finite Automata
- 5 From Regex to a Scanner
- 6 Practice: Tokenizing Code
- 7 Implementation in JavaScript
- 8 Wrap-up

What Is Lexical Analysis?

- Lexical analysis (scanning) is the first phase of a compiler or interpreter.
- It takes in:
 - ▶ a stream of characters from the source file
- It produces:
 - ▶ a stream of tokens such as
 - ▶ keywords (let, if, while)
 - ▶ identifiers (variable and function names)
 - ▶ literals (numbers, strings)
 - ▶ operators and punctuation
- Main job:
 - ▶ group characters into meaningful pieces (lexemes)
 - ▶ give each lexeme a token type
 - ▶ skip whitespace and comments

Where Does Lexical Analysis Fit?

- A very simple view of a compiler or interpreter:
 - ▶ Source program (characters)
 - ▶ → scanner or lexer (tokens)
 - ▶ → parser (syntax trees)
 - ▶ → later phases (semantic checks, code generation, and so on)
- Lexical analysis answers questions like:
 - ▶ Where does each identifier, number, or operator start and end?
 - ▶ What kind of token is each piece of the input?
- In practice, a scanner can be:
 - ▶ hand-written, or
 - ▶ generated from regular expressions and finite automata

Basic Vocabulary

- Character: one position in the file, for example a, +, 7, ;
- Lexeme: a finite sequence of characters from the source program that matches the pattern for a token
- Token: a pair <token-name, attribute> produced by the lexical analyzer to represent a lexeme.
- Example line: `int x = 42;`
- Lexemes in that line:
 - ▶ `int`
 - ▶ `x`
 - ▶ `=`
 - ▶ `42`
 - ▶ `;`
- Tokens for those lexemes:
 - ▶ `int` → token type `INT_KEYWORD`
 - ▶ `x` → token type `IDENT`
 - ▶ `=` → token type `ASSIGN`
 - ▶ `42` → token type `INT_LIT` (value 42)
 - ▶ `;` → token type `SEMICOLON`
- Lexical structure: the rules that say how characters form valid tokens

Example: Token Stream

Consider the JavaScript statement:

```
1 | let sum = a + 10;
```

One possible sequence of tokens:

- token type LET_KW, lexeme let
- token type IDENT, lexeme sum
- token type ASSIGN, lexeme =
- token type IDENT, lexeme a
- token type PLUS, lexeme +
- token type INT_LIT, value 10
- token type SEMICOLON, lexeme ;

Whitespace is usually skipped by the scanner and does not become tokens.

Token Categories in Industry Languages

Common categories in Java, C++, and JavaScript:

- Keywords (reserved words that are not allowed as identifiers)
- Identifiers (names of variables, functions, classes, and so on)
- Literals (integers, floats, strings, chars, booleans)
- Operators (for example, `+` `-` `*` `/` `==` `&&` `——` `!`)
- Separators and punctuation (for example, `;` `,` `(` `)` `{` `}` `[` `]`)
- Comments and whitespace (usually recognized, then ignored)

Java / C++ / JavaScript Token Examples

Category	Java	C++	JavaScript
Keyword	int, class	int, template	let, function
Identifier	myVar	vectorSize	totalSum
Integer literal	42	0, 100	42
String literal	"hi"	"hi"	"hi", 'hi'
Operator	+, ==	+, →	+, ===
Separator	;, {},}	;, (,)	;, (,)
Comment	// ...	// ...	// ...

- 1 Overview and Goals
- 2 Lexical Basics
- 3 Regular Expressions**
- 4 Finite Automata
- 5 From Regex to a Scanner
- 6 Practice: Tokenizing Code
- 7 Implementation in JavaScript
- 8 Wrap-up

Why Use Regular Expressions?

- Many token types follow simple text patterns.
- Examples:
 - ▶ Identifier: letter or underscore, followed by letters, digits, or underscores
 - ▶ Integer literal: one or more digits in a row
 - ▶ Whitespace: one or more spaces, tabs, or newlines
- Regular expressions are a compact way to write these patterns.
- Later, we can turn these patterns into finite automata and then into code.

Regular Languages and Regex (Informal)

A regular language is a set of strings built from simpler pieces using:

- Union (choice): $L_1 \cup L_2$
 - ▶ strings from L_1 or from L_2
- Concatenation: $L_1 L_2$
 - ▶ strings from L_1 followed by strings from L_2
- Kleene star: L^*
 - ▶ zero or more repetitions of strings from L

Regular expressions give us a way to write these ideas:

- $(a|b)$ means either a or b
- ab means a followed by b
- a^* means any number of a, including zero
- Common shorthands: $[0-9]$, $[A-Za-z]$, $\backslash d$, $+$, $?$, and so on

Regex Syntax – Common Matching Symbols

Symbol	Meaning	Example
.	Any character	("..", "a%") – T ("..", ".a") – T ("..", "a") – F
^	Start anchors	("^a.c.", "abcd") – T ("^a", "ac") – F
\$	End anchors	("..cd\$", "abcd") – T ("a\$", "a") – T ("a\$", "aca") – F
[abc]	One of a, b, or c	("^[abc]d", "ad9") – T ("[ab].d\$", "bad") – T ("[ab]x", "cx") – F

Regex Syntax – Common Matching Symbols

Symbol	Meaning	Example
<code>[abc][12]</code>	Can match a, b or c followed by 1 or 2	<code>("[ab][12]", "a2") – T</code> <code>("[ab].[12]", "acd2") – T</code> <code>("[ab][12]", "a3") – F</code>
<code>[^abc]</code>	When ^ is the first character in [], it negates the pattern, matches anything except a, b or c	<code>("[^ab][^12]", "c3#") – T</code> <code>("[^ab]..[^12]", "x123") – T</code> <code>("[^ab][^12]", "c2") – F</code>
<code>[a-e1-8]</code>	Matches ranges between a to e or 1 to 8	<code>("[a-e1-3]", "d#") – T</code> <code>("[a-e1-3]", "2") – T</code> <code>("[a-e1-3]", "f2") – F</code>
<code>xx yy</code>	Matches regex xx or yy	<code>("xx yy", "xx") – T</code> <code>("xx yy", "yy") – T</code> <code>("xx yy", "xy") – F</code>

Regex Syntax – Quantifiers

Regular expression	Description
<code>x?</code>	x occurs once or not at all
<code>X*</code>	X occurs zero or more times (Kleene star)
<code>X+</code>	X occurs one or more times (Kleene cross)
<code>X{n}</code>	X occurs exactly n times
<code>X{n,}</code>	X occurs n or more times
<code>X{n,m}</code>	X occurs at least n times but not more than m times

Regex Syntax – Metacharacters

Regular expression	Description
<code>\d</code>	Any digit, short for <code>[0-9]</code>
<code>\D</code>	Any non-digit, short for <code>[^0-9]</code>
<code>\s</code>	Any whitespace or delimiter character, short for <code>[\t\n\x0B\f\r]</code>
<code>\S</code>	Any non-whitespace character, short for <code>[^\s]</code>
<code>\w</code>	Any word character, short for <code>[a-zA-Z_0-9]</code>
<code>\W</code>	Any non-word character, short for <code>[^\w]</code>
<code>\b</code>	A word boundary.
<code>\B</code>	A non word boundary.

Practice 1: Regex – Match or No Match?

Pattern: `[A-Za-z_][A-Za-z0-9_]*`

For each string, decide: match or no match?

- `x`
- `_main`
- `9lives`
- `hello_world`
- `user-name`

What rule does this regex encode for identifiers?

Example to try: `node regex.js`

Regex Examples for Tokens

Assume ASCII letters, digits, underscore:

- Identifier (simple version): `[A-Za-z_][A-Za-z0-9_]*`
- Integer literal (non-negative): `[0-9]+`
- Whitespace: `[\t ]+`
- Line comment (C++/Java/JS style): `//.*`

Real languages add more details (hex, floats, Unicode, and so on), but the basic idea is the same.

- 1 Overview and Goals
- 2 Lexical Basics
- 3 Regular Expressions
- 4 Finite Automata**
- 5 From Regex to a Scanner
- 6 Practice: Tokenizing Code
- 7 Implementation in JavaScript
- 8 Wrap-up

Finite Automata: Intuition

- A finite automaton (FA) is a very simple machine with:
 - ▶ a small set of states
 - ▶ a start state
 - ▶ one or more accepting (final) states
 - ▶ rules that say how to move between states based on the next character
- How it works:
 - ▶ start in the initial state
 - ▶ read characters one at a time and follow the transitions
 - ▶ if you end in an accepting state, the input string is accepted
- Regular expressions and finite automata describe the same class of languages.

Deterministic vs Non-deterministic Automata

- In a deterministic finite automaton (DFA):
 - ▶ for each state and each input character, there is at most one next state
- In a non-deterministic finite automaton (NFA):
 - ▶ there may be several possible next states
 - ▶ there may be moves on empty input (epsilon transitions)
- The important fact for us:
 - ▶ every NFA can be converted into an equivalent DFA
 - ▶ scanners are usually implemented as DFAs for speed

Deterministic Finite Automation for Integer Literals

Target language: non-empty sequences of digits, described by $[0-9]^+$.

States (informal):

- q_0 : start, no digits seen yet
- q_1 : have seen at least one digit (accepting)
- q_{dead} : anything invalid

Behavior:

- From q_0 :
 - ▶ digit goes to q_1
 - ▶ non-digit goes to q_{dead}
- From q_1 :
 - ▶ digit stays in q_1
 - ▶ non-digit means we stop and accept the digits we saw as one token

This is the basic idea of how a scanner finds where an integer literal ends.

Example: Running the Integer DFA

Target pattern: sequences of digits $[0-9]^+$.

Example 1: input "42"

- Start in q_0
- Read '4' (digit): $q_0 \rightarrow q_1$
- Read '2' (digit): $q_1 \rightarrow q_1$
- End of input in accepting state q_1
- Result: one integer token with lexeme "42"

Example 2: input "123x"

- Start in q_0
- Read '1', '2', '3' (digits): $q_0 \rightarrow q_1 \rightarrow q_1 \rightarrow q_1$
- Next character is 'x' (not a digit)
- DFA says: stop token before 'x'
- Result: integer token "123", remaining input "x" for the next token

DFA Sketch for Identifiers

Simple identifier rule: first character is a letter or underscore, rest are letters, digits, or underscores.

States:

- q_0 start, seen nothing yet
- q_{id} seen a valid first character (accepting)
- q_{dead} for anything that can not start an identifier

Behavior:

- From q_0 :
 - ▶ letter or underscore goes to q_{id}
 - ▶ anything else goes to q_{dead}
- From q_{id} :
 - ▶ letter, digit, or underscore stays in q_{id}
 - ▶ other character means end of identifier token

Example: Running the Identifier DFA 1

Simple identifier rule:

- first character: letter or underscore
- rest: letters, digits, or underscores

Example 1: input "x1y"

- Start in q_0
- Read 'x' (letter): $q_0 \rightarrow q_{id}$
- Read '1' (digit): stay in q_{id}
- Read 'y' (letter): stay in q_{id}
- End of input in accepting state q_{id}
- Result: identifier token with lexeme "x1y"

Example: Running the Identifier DFA 2

Simple identifier rule:

- first character: letter or underscore
- rest: letters, digits, or underscores

Example 2: input "_main3"

- Start in q_0
- Read '_': $q_0 \rightarrow q_{id}$
- Read 'm','a','i','n','3': stay in q_{id} each time
- End of input in q_{id}
- Result: identifier token with lexeme "_main3"

- 1 Overview and Goals
- 2 Lexical Basics
- 3 Regular Expressions
- 4 Finite Automata
- 5 From Regex to a Scanner**
- 6 Practice: Tokenizing Code
- 7 Implementation in JavaScript
- 8 Wrap-up

From Regex to Scanner (High Level)

- For each token type:
 - ▶ write a regular expression
 - ▶ turn that regular expression into an automaton
- Combine the automata for all token types into one machine.
- Use that machine to scan the input:
 - ▶ find the longest prefix that matches some token
 - ▶ output that token
 - ▶ then continue with the rest of the input
- Tools like lex, flex, and JLex can do this automatically.

Manual Lexing Walkthrough

Consider the line:

```
1 | let total = a + 10;
```

Walk through as a scanner would:

- Read `l e t` → matches keyword pattern, token `LET_KW`
- Skip the space
- Read `t o t a l` → matches identifier pattern, token `IDENT`
- Skip the space
- Read `=` → matches assign pattern, token `ASSIGN`
- Skip the space
- Read `a` → token `IDENT`
- Skip the space
- Read `+` → token `PLUS`
- Skip the space
- Read `1 0` → token `INT_LIT`
- Read `;` → token `SEMICOLON`

- 1 Overview and Goals
- 2 Lexical Basics
- 3 Regular Expressions
- 4 Finite Automata
- 5 From Regex to a Scanner
- 6 Practice: Tokenizing Code**
- 7 Implementation in JavaScript
- 8 Wrap-up

Practice 2: Tokenizing Java (Question)

Java snippet:

```
1 | int sum = 0;  
2 | for (int i = 0; i < 10; i = i + 1) {  
3 |     sum = sum + i;  
4 | }
```

Questions:

- What are the lexemes for the first line?
- Which token types do they map to?
- Where do you see keywords, identifiers, literals, operators, separators in this whole snippet?

Practice 2: Tokenizing Java (Answer 1)

Java snippet:

```
1 | int sum = 0;
```

- Lexemes for the first line:

- ▶ `int`
- ▶ `sum`
- ▶ `=`
- ▶ `0`
- ▶ `;`

- Token types (one possible naming scheme):

- ▶ `INT_KW`, `IDENT`, `ASSIGN`, `INT_LIT`, `SEMICOLON`

Practice 2: Tokenizing Java (Answer 2)

Java snippet:

```
1 | int sum = 0;  
2 | for (int i = 0; i < 10; i = i + 1) {  
3 |     sum = sum + i;  
4 | }
```

- Keywords: int, for

- Identifiers: sum, i

- Operators:

 - ▶ =

 - ▶ +

 - ▶ <

- Separators:

 - ▶ (

 - ▶)

 - ▶ {

 - ▶ }

 - ▶ ;

- 1 Overview and Goals
- 2 Lexical Basics
- 3 Regular Expressions
- 4 Finite Automata
- 5 From Regex to a Scanner
- 6 Practice: Tokenizing Code
- 7 Implementation in JavaScript**
- 8 Wrap-up

Scanner Responsibilities

- Read input characters from a source file or string.
- Group characters into tokens using lexical rules.
- Return a sequence (or stream) of tokens to the parser.
- Skip over things that should not become tokens:
 - ▶ whitespace
 - ▶ comments
- Keep track of:
 - ▶ line numbers and columns, for error messages
 - ▶ the original lexeme for identifiers, literals, and so on

Design Choices

- How to write the scanner:
 - ▶ a table-driven DFA, or
 - ▶ hand-written if and else or switch code
- Lookahead:
 - ▶ sometimes we must peek at the next character
 - ▶ examples: = versus ==, + versus ++
- Error handling:
 - ▶ what should we do with unexpected characters?
 - ▶ one simple strategy: report the error and maybe skip one character

A Tiny Token Definition (TypeScript)

```
1  const TokenType = {
2    LET: "LET",
3    IDENT: "IDENT",
4    NUMBER: "NUMBER",
5    OP: "OP",
6    ASSIGN: "ASSIGN",
7    SEMI: "SEMI"
8  } as const;
9  class Token {
10    public type: TokenType;
11    public lexeme: string;
12    constructor(type: TokenType, lexeme: string) {
13      this.type = type;
14      this.lexeme = lexeme;
15    }
16    toString(): string {
17      return `${this.type}(${this.lexeme})`;
18    }
19  }
```

Regex Specification for Tokens (TypeScript)

```
1  const tokenSpecs: [RegExp, TokenType | null][] = [  
2    // Whitespace (skip)  
3    [/^\s+/, null],  
4    // Single-line comments (skip)  
5    [/^\s+\/.*\/, null],  
6    // Keywords  
7    [/^let\b/, TokenType.LET],  
8    // Identifiers and numbers  
9    [/^[A-Za-z_][A-Za-z0-9_]*/, TokenType.IDENT],  
10   [/^[0-9]+/, TokenType.NUMBER],  
11   // Operators and punctuation  
12   [/^[+ \- * \/ ]/, TokenType.OP],  
13   [/^=/, TokenType.ASSIGN],  
14   [/^;/, TokenType.SEMI]  
15 ];
```

- Each entry is a pair: a regular expression and a token type.
- If the token type is null, the scanner skips that text.

Tiny Lexer in TypeScript

```
1 function lex(input: string): Token[] {
2     const tokens: Token[] = [];
3     let str: string = input;
4     while (str.length > 0) {
5         let match: RegExpExecArray | null = null;
6         let matchedType: TokenType | null = null;
7         for (const [regex, type] of tokenSpecs) {
8             match = regex.exec(str);
9             if (match) {
10                 matchedType = type;
11                 break;
12             }
13         }
14         if (!match) {
15             throw new Error(`Unexpected character: '${str[0]}'`);
16         }
17         const lexeme = match[0];
18         if (matchedType !== null) {
19             tokens.push(new Token(matchedType, lexeme));
20         }
21         str = str.slice(lexeme.length);
22     }
23     return tokens;
24 }
```

Running the Tiny Lexer

```
1 | const src = "let sum = a + 10;";  
2 |  
3 | const tokens = lex(src);  
4 | for (const t of tokens) {  
5 |   console.log(t.toString());  
6 | }
```

Expected output (in words):

- a LET token with lexeme let
- an IDENT token with lexeme sum
- an ASSIGN token with lexeme =
- an IDENT token with lexeme a
- an OP token with lexeme +
- a NUMBER token with lexeme 10
- a SEMI token with lexeme ;

Example: `npx ts-node tiny_lexer.ts "let sum = a + 10;"`

Handling Whitespace and Comments

- What counts as whitespace in most languages?
 - ▶ space character, tab character, newline, carriage return
- Typical scanner rule:
 - ▶ recognize one or more whitespace characters
 - ▶ consume them (advance the input position)
 - ▶ do not return a token for them
- Why we still care about whitespace:
 - ▶ we must update line and column numbers
 - ▶ newlines affect error messages and sometimes layout
- Comments are handled the same way:
 - ▶ recognize the comment pattern (for example, `//...` or `/* ... */`)
 - ▶ consume everything up to the end of the comment
 - ▶ return no token, but still update line and column if there are newlines
- Exception examples:
 - ▶ Python: indentation (leading spaces) is part of the syntax
 - ▶ some languages treat end of line as a special token

- 1 Overview and Goals
- 2 Lexical Basics
- 3 Regular Expressions
- 4 Finite Automata
- 5 From Regex to a Scanner
- 6 Practice: Tokenizing Code
- 7 Implementation in JavaScript
- 8 Wrap-up**

Error Reporting in Scanners

- Real source code will contain mistakes.
- Your scanner should detect bad input such as:
 - ▶ characters that are not allowed in the language
 - ▶ badly formed tokens, if you choose to support more complex tokens
- When an error is found:
 - ▶ print a clear message with the line and column
 - ▶ show the unexpected character or lexeme

Connecting Back to Java/C++ Token Categories

- Java and C++ use the same basic ideas:
 - ▶ keywords: int, class, if, while, and so on
 - ▶ identifiers: names that follow the language rules
 - ▶ literals: integers, floats, chars, strings, booleans
 - ▶ operators: arithmetic, comparison, logical, bitwise
 - ▶ separators: punctuation and delimiters
- Industrial compilers have many more token types and details.
- The small scanners you build in this course use the same core ideas, just on a smaller scale.

Summary

By the end of Week 2, you should be able to:

- say what characters, lexemes, and tokens are
- list the main token categories in common languages
- write simple regular expressions for token patterns
- explain, at a high level, what a finite automaton does
- read and write a small scanner in JavaScript or Python

Next week: from tokens to parse trees, and the basics of parsing.